

**Автономная некоммерческая организация высшего образования
«Университет Иннополис»**

**АННОТАЦИЯ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
(БАКАЛАВРСКУЮ РАБОТУ)
ПО НАПРАВЛЕНИЮ ПОДГОТОВКИ
09.03.01 «Информатика и вычислительная техника»**

**НАПРАВЛЕННОСТЬ (ПРОФИЛЬ) ОБРАЗОВАТЕЛЬНОЙ
ПРОГРАММЫ
«Информатика и вычислительная техника»**

Тема **Разработка системы мониторинга производительности и
обнаружения регрессий в Android-приложениях**

Выполнил

**Миннемуллин Наиль
Фидаилевич**



ПОДПИСЬ

Иннополис, 2026

Содержание выпускной квалификационной работы

1	Введение	7
1.1	Предметная область и мотивация	7
1.2	Исследовательский пробел	8
1.3	Исследовательские вопросы и цель работы	8
1.4	Новизна и вклад	9
1.5	Структура работы	10
2	Обзор литературы	11
2.1	Мониторинг производительности	12
2.2	Подходы к сбору данных о производительности	12
2.3	Метрики производительности	16
2.4	Обнаружение регрессий производительности	17
2.5	Выводы	20
3	Методология	22
3.1	Требования и ограничения системы	22
3.2	Высокоуровневая архитектура	23
3.3	Проектирование мобильного SDK	24
3.4	Проектирование backend-сервиса	28
3.5	Выводы	33
4	Реализация	34
4.1	Реализация мобильного SDK	34
4.2	Реализация backend-сервиса	41
4.3	Реализация обнаружения регрессий	44
4.4	Реализация frontend-дашборда	46
4.5	Выводы	50
5	Оценка и обсуждение	51
5.1	Установка валидации	51
5.2	Оценка накладных расходов SDK	52
5.3	Сквозное тестирование системы	54
5.4	Обсуждение результатов	58
5.5	Ограничения и направления дальнейшей работы	60
6	Заключение	62
	Список использованной литературы	64

Введение

Каждый новый релиз мобильного приложения может незаметно ухудшать производительность на различных экранах, например через добавление нового функционала или обновление внешних зависимостей. На Android такие регрессии особенно трудно выявить, поскольку устройства пользователей сильно различаются по характеристикам и версиям операционной системы, и один и тот же релиз может вести себя по-разному на разных устройствах. Автоматическое обнаружение регрессий хорошо изучено для серверных систем [1], однако существующие инструменты для Android с поддержкой оповещений в основном опираются на пороги, задаваемые разработчиком вручную, и сравнивают все устройства вместе, не учитывая различий в их производительности [2].

Цель работы состоит в разработке системы, которая непрерывно собирает метрики производительности с Android-устройств пользователей и автоматически обнаруживает регрессии между релизами с учётом различных классов производительности устройств. Ключевая новизна состоит в том, что релизы сравниваются не по совокупности всех устройств, а внутри отдельных когорт со схожими характеристиками. Слабые устройства обычно составляют меньшую долю собранных данных. При сравнении по всем устройствам сразу регрессия, заметная только на слабых устройствах, теряется на фоне данных с более производительных. Однако поддержание производительности на таких устройствах не менее важно [3].

Задачи работы. (1) Проанализировать существующие подходы к мониторингу и обнаружению регрессий на Android. (2) Спроектировать архитектуру системы из SD для Android, backend-сервиса и пользовательского интерфейса. (3) Реализовать на устройстве сбор метрик производительности с низкими накладными расходами. (4) Реализовать метод обнаружения регрессий, учитывающий различные классы устройств. (5) Экспериментально подтвердить работоспособность системы, оценить накладные расходы SDK на устройствах и показать эффективность стратификации по когортам.

Краткое описание основной части работы

Обзор литературы (глава 2). Во второй главе анализируются подходы к сбору данных о производительности на Android, которые делятся на предрелизное профилирование и пострелизный мониторинг, а также методы автоматического обнаружения регрессий [4], [5]. Рассматривается фрагментация устройств Android [3] и схемы их сегментации по производительности. Выявленный пробел состоит в отсутствии автоматического релиз-парного обнаружения регрессий, стратифицированного по когортам устройств.

Методология (глава 3). В третьей главе формулируются требования к системе и описывается её архитектура из четырёх компонентов. Мобильный SDK на Kotlin собирает метрики UI, время холодного старта, загрузку CPU и объём используемой памяти. Backend-сервис на Ktor принимает батчи метрик, хранит реляционные данные в PostgreSQL, временные ряды в ClickHouse [6] и относит устройства к когортам Low, Medium и High. Сервис обнаружения регрессий для каждой пары последовательных релизов вычисляет относительный сдвиг P95 в разрезе метрики, экрана и когорты и фиксирует регрессию при превышении порога в 15 процентов.

Реализация (глава 4). В четвёртой главе описывается реализация компонентов. SDK написан на Kotlin с буферизацией метрик в Room и пакетной передачей по сети, а backend на Ktor с авторизацией через JWT и BCrypt. Дашборд построен на Streamlit. Сервис обнаружения регрессий реализован на Python, использует агрегирующие запросы ClickHouse и отправляет уведомления в Telegram. Все компоненты упакованы в Docker и развёрнуты как открытая система.

Оценка и обсуждение (глава 5). В пятой главе система проверяется экспериментально. Накладные расходы SDK не превысили 5,2 процентных пункта загрузки CPU, 5,4 МБ (4,1 процента) оперативной памяти и 75 мс (5,1 процента) времени холодного старта. Тестирование системы подтвердило корректность её работы. Также разделение на когорты оказалось полезным: регрессия, проявляющаяся только на слабых устройствах, была успешно обнаружена.

Заключение

Результатом работы стала спроектированная, реализованная и экспериментально проверенная система мониторинга производительности Android-приложений и автоматического обнаружения регрессий между релизами. Последовательные релизы сравниваются не по всем устройствам сразу, а независимо внутри когорт Low, Medium и High, что позволяет находить регрессии, проявляющиеся только на определённом классе устройств.

Эмпирически показано, что SDK сохраняет работоспособность при низких накладных расходах. В худшем случае на тестовых устройствах прирост загрузки CPU не превысил 5,2 процентных пункта, использования памяти 5,4 МБ (4,1 процента), а времени холодного старта 75 мс (5,1 процента). Сквозное тестирование подтвердило, что система корректно фиксирует регрессии. В эксперименте с регрессией, чувствительной к классу устройства, разделение на когорты продемонстрировало свою эффективность: регрессия, заметная только на слабых устройствах, была успешно обнаружена, тогда как при сравнении по всем устройствам сразу она бы затерялась на фоне данных с более производительных устройств.

Тем самым работа закрывает обозначенный пробел в области автоматического пострелизного обнаружения регрессий на Android и предлагает готовую к использованию систему со стратификацией сравнения релизов по когортам устройств. Перспективы дальнейшей работы включают расширение набора метрик, а также улучшение алгоритмов разделения на когорты и обнаружения регрессий.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Netflix Technology Blog. “Automated Canary Analysis at Netflix with Kayenta.” (2018), url: <https://netflixtechblog.com/automated-canary-analysis-at-netflix-with-kayenta-3260bc7acc69> (дата обр. 09.06.2026).
- [2] Firebase. “Firebase Performance Monitoring.” Accessed: 30 Oct 2025. (2025), url: <https://firebase.google.com/docs/perf-mon>.
- [3] Z. Dong, Y. Zhao, T. Liu и др., “Same App, Different Behaviors: Uncovering Device-specific Behaviors in Android Apps,” в *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, сер. ASE ’24, Sacramento, CA, USA: Association for Computing Machinery, 2024, с. 2099—2109, ISBN: 9798400712487. DOI: 10.1145/3691620.3695272.
- [4] T. H. D. Nguyen, B. Adams, Z. M. Jiang, A. E. Hassan, M. Nasser и P. Flora, “Automated detection of performance regressions using statistical process control techniques,” в *Proceedings of the 3rd ACM/SPEC International Conference on Performance Engineering (ICPE ’12)*, ACM, 2012, с. 299—310.
- [5] D. Y. Yoon, Y. Wang, M. Yu и др., “FBDetect: Catching Tiny Performance Regressions at Hyperscale through In-Production Monitoring,” в *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP ’24)*, ACM, 2024.
- [6] R. Schulze, T. Schreiber, I. Yatsishin, R. Dahimene и A. Milovidov, “ClickHouse - Lightning Fast Analytics for Everyone,” *Proceedings of the VLDB Endowment*, т. 17, № 12, с. 3731—3744, 2024. DOI: 10.14778/3685800.3685802.